0.14 Graphen

0.14.1 Einleitung

0.14.1.1 Graphen

Definitionen:

- Graph G: Zusammenfassung einer Menge V und einer Relation $E \subseteq V \times V$ zu einer Struktur $G = (V, E)$
- $v \in V$: *Knoten* (*node*, *vertex*)
- $e = (v, w) \in E$: *Kante* (*edge*, *arc*) zwischen zwei Knoten $v \in V$ und $w \in V$
- Kante (v, w) existiert: Knoten v, w sind *adjazent* (benachbart)
- $|V|$: Anzahl der Knoten
- $|E|$: Anzahl der Kanten
- $|V|$ und $|E|$ sind die Parameter für die Bestimmung der Problemgröße bei der Laufzeitabschätzung

0.14.1.2 Beispiele

Beispiel: Derselbe Graph in zwei unterschiedlichen Darstellungen

Abbildung 108: Beispiel für unterschiedliche Darstellung desselben Graphen

- Die geometrische Anordnung der Knoten und Kanten in der Darstellung ist ohne Bedeutung
- Wichtig ist, welche Paare von Knoten in welcher Richtung miteinander verbunden sind

© R.-R. Grigat 2003-2019

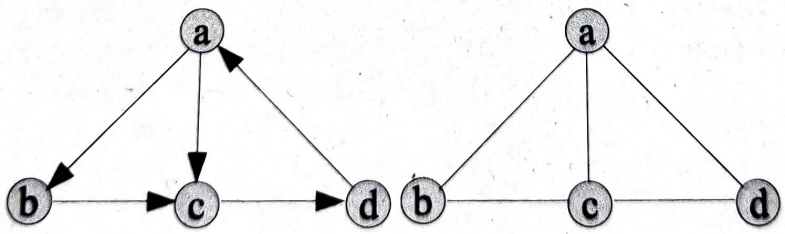
0.14.1.3 Klassifizierung von Graphen

Kriterien zur Klassifizierung von Graphen:

- symmetrisch vs. asymmetrisch
  - symmetrisch: $\forall v, w \in V,\ v \neq w$ gilt:
    Kante (v, w) existiert $\Rightarrow$ Kante (w, v) existiert
  - asymmetrisch: $\exists v, w \in V,\ v \neq w$ mit:
    Kante (v, w) existiert $\wedge$ Kante (w, v) existiert *nicht*
- reflexiv vs. irreflexiv
  - reflexiv: wenn jeder Knoten mit sich selbst verbunden ist
  - irreflexiv: wenn die Existenz der Kante (v, v) für alle Knoten ausgeschlossen ist
- transitiv: $\forall v, w, z \in V$ gilt: Kanten (v, w) und (w, z) existieren $\Rightarrow$ Kante (v, z) existiert

0.14.1.4 Gerichtete und ungerichtete Graphen (Beispiel)

Beispiel



(a) Gerichteter Graph (b) Ungerichteter Graph

Abbildung 109: Gerichtete vs. ungerichtete Graphen

© R.-R. Grigat 2003-2019

0.14.1.5 Klassifizierung von Graphen

Kriterien zur Klassifizierung von Graphen:

- gerichtet vs. ungerichtet
  - Gerichteter Graph (Digraph, für **directed graph**)
    - Kanten besitzen Richtung
    - Darstellung der Kanten durch Pfeile
    - symmetrisch oder asymmetrisch
    - Schleifen (v,v) von Knoten zu sich selbst sind erlaubt
  - Ungerichteter Graph:
    - *Symmetrischer* Graph, also Spezialfall
    - Kanten verbinden zwei Knoten jeweils in beiden Richtungen
    - Kanten $\{(v,w),(w,v)\}$ werden durch *eine* Kante (v,w) oder (w,v) repräsentiert
    - Schleifen (v,v) *nicht* erlaubt
- Zunächst werden nun Algorithmen für gerichtete Graphen, also den allgemeineren Fall, diskutiert
- Anschließend (Abschnitt **??**) folgen ungerichtete Graphen

0.14.2 Digraphen

0.14.2.1 Beispiel zu Digraphen

Beispiel

$$V = \{a, b, c, d\}$$
$$E = \{(a,b),(a,c),(b,c),(c,d),(d,a)\}$$

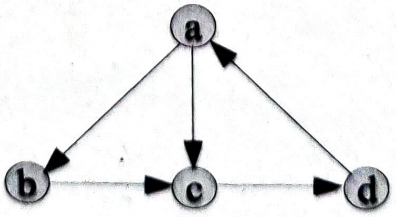


Abbildung 110: Ein gerichteter Graph

© R.-R. Grigat 2003-2019

0.14.2.2 Definitionen und Terminologie

Definitionen:

- $(v,w) \in E \Rightarrow$ v ist *Vorgängerknoten* von w, w ist *Nachfolgerknoten* von v
- *Grad* eines Knotens: Anzahl eingehender und ausgehender Kanten
  - *Eingangsgrad*: Anzahl eingehender Kanten
  - *Ausgangsgrad*: Anzahl ausgehender Kanten
- *Pfad*, *Weg*: Folge von Knoten $v_1, \ldots, v_n$, so dass $(v_i, v_{i+1}) \in E \quad \forall_i \, 1 \leq i \leq n-1$
- *Länge* eines Pfades: Anzahl der Kanten des Pfades
- *Einfacher* Pfad: Alle Knoten eines Pfades sind paarweise verschieden (Ausnahme: $v_1 = v_n$ erlaubt)
- *Zyklus*: Einfacher Pfad, bei dem $v_1 = v_n$ mit $n > 1$

Beispiel:
Digraph in Abb. 110:

- Eingangsgrad des Knoten c: 2
- Ausgangsgrad des Knoten c: 1
- Ein Pfad: (a,b,c,d)
- Ein Zyklus: (a,b,c,d,a)

0.14.2.3 Gewichtete Graphen

Definition Gewichteter Graph

- Ein Graph, dessen Kanten Werte (*Gewichte*) zugeordnet sind
- Beispiele:
  - Kosten wie Entfernungen oder Reisezeiten in einem Verkehrsnetz
  - Ströme bzw. Spannungen in elektrischem Netzwerk

© R.-R. Grigat 2003-2019

Beispiel

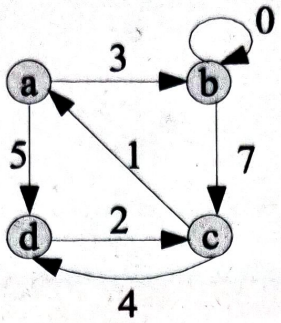


Abbildung 111: Ein gewichteter Graph

0.14.2.4 Adjazenzmatrix

Definition Adjazenzmatrix

- Darstellung des Digraphen $D = (V, E)$ mit $V = \{v_1, v_2, \ldots, v_n\}$
- Boolesche $n \times n$ Matrix A mit

$$a_{ij} = \begin{cases} 1, & (v_i, v_j) \in E \\ 0, & \text{sonst} \end{cases}$$

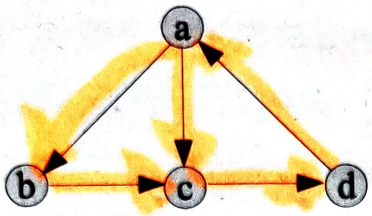
- engl. adjacence $\longleftrightarrow$ Nachbarschaft

Darstellung eines Graphen mit Adjazenzmatrix

- Speicheraufwand: $O(n^2)$, $n = |V|$
- Rechenzeit, um Adjazenz zweier Knoten zu testen: $O(1)$

Beispiel

© R.-R. Grigat 2003-2019



(a) Digraph

	a	b	c	d
a	0	1	1	0
b	0	0	1	0
c	0	0	0	1
d	1	0	0	0

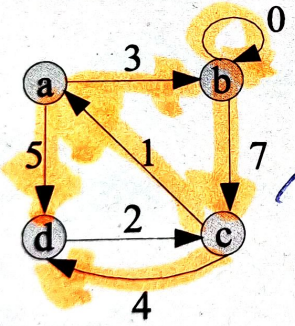
(b) Adjazenzmatrix

Abbildung 112: Digraph und Adjazenzmatrix

Adjazenzmatrix eines gewichteten Graphen

- die Gewichte als Matrixelemente
- das Fehlen einer Kante durch ∞ gekennzeichnet
  - falls nur positive Gewichte auftreten, kann nicht vorhandene Kante auch mit 0 repräsentiert werden

Beispiel



(a) Digraph

	a	b	c	d
a	∞	3	∞	5
b	∞	0	7	∞
c	1	∞	∞	4
d	∞	∞	2	∞

(b) Adjazenzmatrix

Abbildung 113: Gewichteter Graph und Adjazenzmatrix

0.14.2.5 Adjazenzlisten

Definition Adjazenzliste

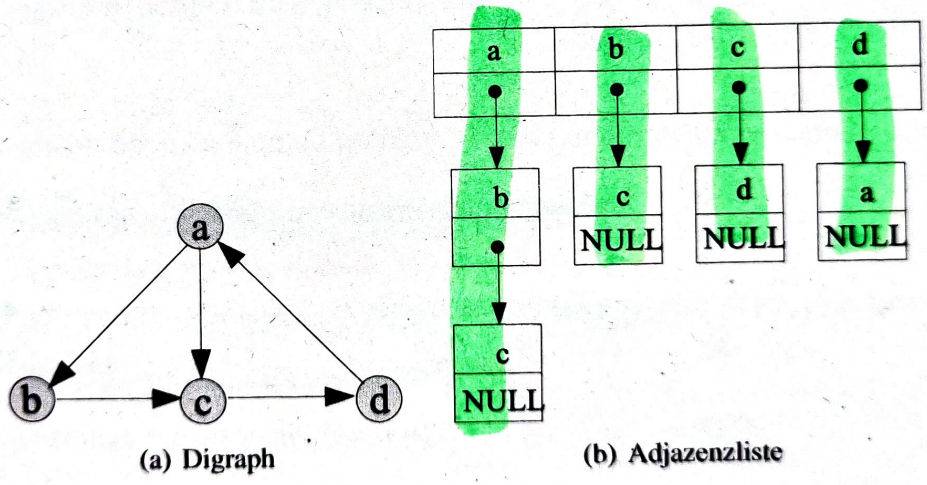
- Liste aller Nachfolgerknoten eines Knotens
- Reihenfolge der Nachfolgerknoten beliebig

© R.-R. Grigat 2003-2019

Darstellung eines Graphen mit Adjazenzlisten

- Für jeden Knoten des Graphen eine Adjazenzliste anlegen
- Alle Adjazenzlisten in einer Liste verwalten
- Speicheraufwand: $O(n+e)$ mit $n = |V|$ und $e = |E| \leq n^2$
- Rechenzeit, um Adjazenz zweier Knoten zu testen: $O(n)$
- Adjazenzlisten vorteilhaft, wenn $e << n^2$

Beispiel



(a) Digraph (b) Adjazenzliste

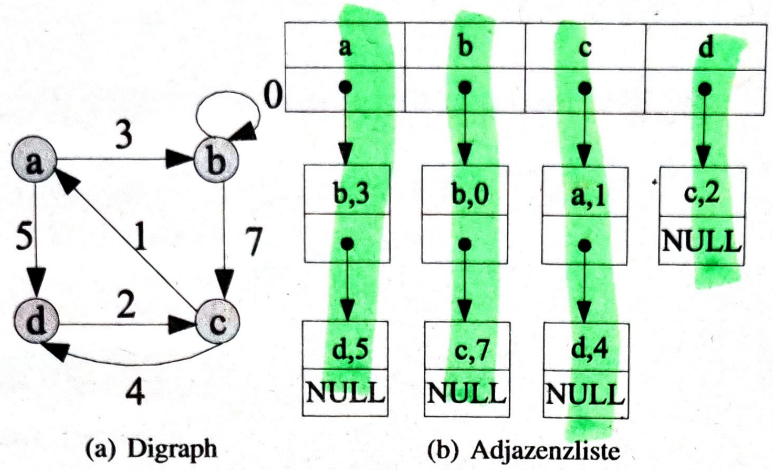
Abbildung 114: Digraph und Adjazenzliste

Adjazenzliste eines gewichteten Graphen

- Gewicht $w(u,v)$ wird zusammen mit Knoten v in Adjazenzliste von u gespeichert

Beispiel

© R.-R. Grigat 2003-2019



(a) Digraph (b) Adjazenzliste

Abbildung 115: Gewichteter Digraph und Adjazenzliste

© R.-R. Grigat 2003-2019

0.14.3 Durchsuchen

0.14.3.1 Durchsuchen (Traversierung) von Graphen

Systematisch alle Knoten besuchen:

- *Tiefensuche* (depth-first search)
  - erzeugt *Tiefensuche-Spannbäume*
  - Suche Zyklen (via Rückwärtskanten)
  - Suche stark zusammenhängende Komponenten
- *Breitensuche* (breadth-first search)
  - erzeugt *Breitensuche-Spannbäume*
  - Suche kürzeste Wege vom Wurzelknoten aus (ähnlich Algorithmen von Dijkstra, Prim, siehe unten)
  - Prüfe Erreichbarkeit vom Wurzelknoten aus
- Die neuen, oben genannten Begriffe werden nun schrittweise eingeführt.

0.14.3.2 Wurzelgraphen und Teildigraphen

Definition Wurzelgraph
Sind alle Knoten von einem Knoten $r \in V$ aus erreichbar, so heißt der Graph *Wurzelgraph*. r heißt *Wurzelknoten* des Wurzelgraphen.

Definition Teildigraph
Ein Digraph $D' = (V', E')$ ist *Teildigraph* des Digraphen $D = (V, E)$, wenn $V' \subseteq V$ und $E' \subseteq E$

0.14.3.3 Spannbäume (Erzeugende Bäume, Spanning Trees)

Definition Spannbaum (erzeugender Baum) eines Wurzelgraphen mit Wurzelknoten $r \in V$

- zyklenfreier Teildigraph $D' = (V, E')$ des Wurzelgraphen
- enthält alle Knoten aus V, aber nur so viele Kanten aus E, wie für Baumdarstellung notwendig

© R.-R. Grigat 2003-2019

- Knoten des Spannbaums haben Eingangsgrad 1, Ausnahme: Wurzel des Spannbaums hat Eingangsgrad 0

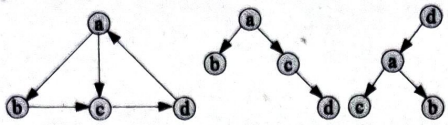


Abbildung 116: Digraph 116(a) und Alternativen für Spannbäume 116(b)-116(c), jeder Knoten ist potentieller Wurzelknoten von 116(a)

0.14.3.4 Erzeugender Wald

Definition Erzeugender Wald eines allgemeinen Graphen

- Digraph $D = (V,E)$ sei kein Wurzelgraph (also von keinem Knoten aus sind alle Knoten erreichbar)
- Digraph lässt sich vollständig zerlegen in disjunkte Wurzelgraphen
- Für jeden Wurzelgraphen wird ein Spannbaum definiert, also ein zyklenfreier Teildigraph $D' = (V',E')$
- Die Menge der Spannbäume aller Teildigraphen enthält alle Knoten aus V und heißt *erzeugender Wald*

0.14.3.5 Tiefensuche

Algorithmus zur Tiefensuche

```
1  void Tiefensuche(Knoten v)
2  {
3    if (v noch nicht besucht)
4    {
5      markiere v als besucht;
6      for (int i=1; i<= Anzahl_Nachfolger(v); i++)
7      {
8        Tiefensuche(v.Nachfolger[i]);  // Rekursiver Aufruf
9      }
10   }
11 }
```

© R.-R. Grigat 2003-2019

```
1 for (int i=1; i<=n; i++)
2 {
3   Tiefensuche(Knoten[i]);  // es könnten noch Knoten unbearbeitet sein
4 }
```

- **for**-Schleife über alle Knoten notwendig, da i. A. nicht alle Knoten von jedem Wurzelkandidaten aus erreichbar sind

0.14.3.6 Beispiel zur Tiefensuche

Beispiel: Wurzelknoten a und f

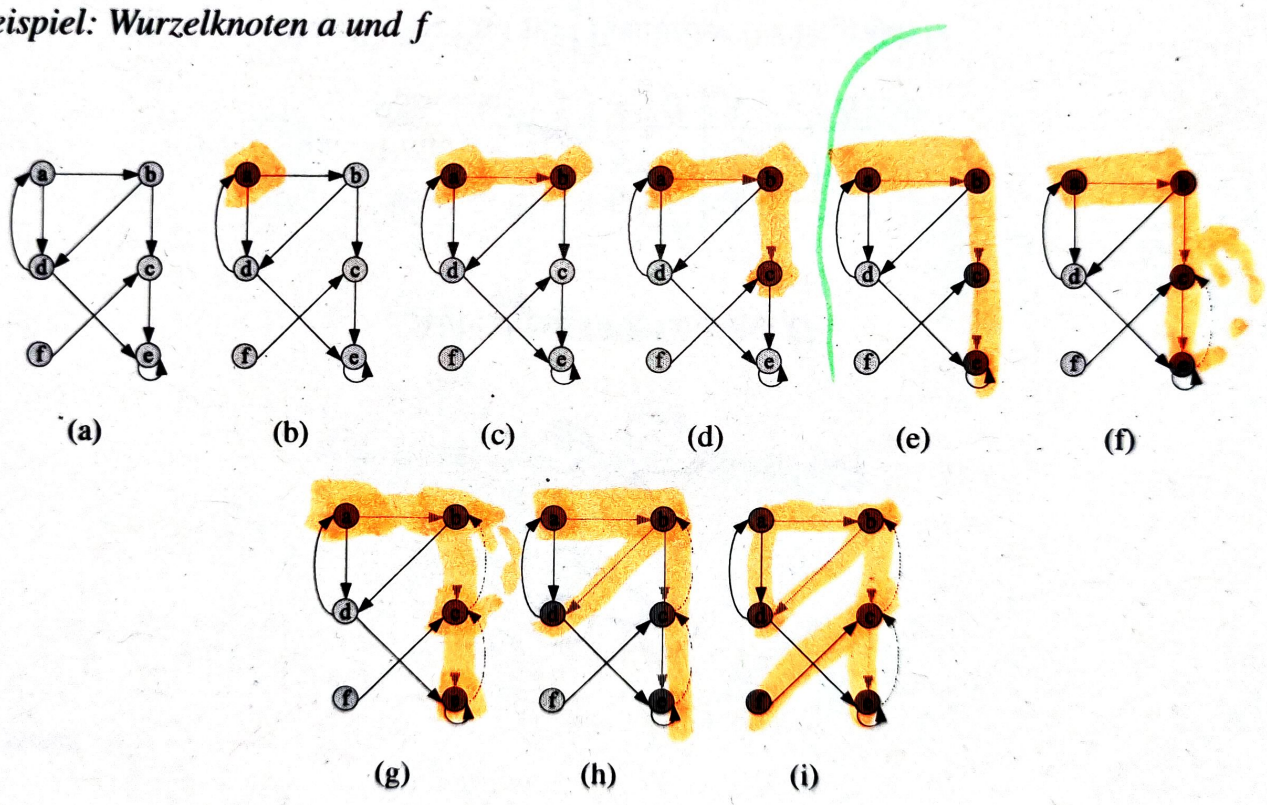


Abbildung 117: Tiefensuche

- Aufsuchen des Wurzelknotens *a*
- Aufsuchen des Nachfolgerknotens *b*
- Aufsuchen des Nachfolgerknotens *c*
- Aufsuchen des Nachfolgerknotens *e*kein Nachfolger: Ende des Weges
- Knoten rückwärts: *c* dort kein unbesuchter Nachfolgerknoten

© R.-R. Grigat 2003-2019

- Knoten rückwärts: b
- Aufsuchen des Nachfolgerknotens $d$keine unbesuchten Vorgänger-/Nachfolgerknoten
- Neuen Wurzelknoten suchen: f, von dort aus keine weiteren unbesuchten Knoten
- Kein weiterer unbesuchter Wurzelknoten

0.14.3.7 Präorder und Postorder bei Tiefensuche

- *Präorder:* Nummer der Reihenfolge, in der der Knoten v innerhalb einer Tiefensuche erreicht wird
- *Postorder:* Nummer der Reihenfolge, in der für den Knoten die Tiefensuche nach Durchlaufen aller Nachfolger beendet wird, d.h. nach Vergabe der Postorder (nur) aller direkten und weiter entfernten Nachfolger *im Spannbaum*
- $\mathit{Präorder}(v) = 0 \Longleftrightarrow$ Knoten v noch nicht besucht
- $\mathit{Postorder}(v) = 0 \Longleftrightarrow$ noch *nicht* alle direkten und weiter entfernten Nachfolgerknoten von v (nur) im Spannbaum wurden bereits mit Postorder versehen
- Zur Bestimmung der Postorder muss die Postorder sämtlicher Nachfolger im Spannbaum geeignet verwaltet werden (z.B. in Adjazenzmatrix)

0.14.3.8 Kantenklassifizierung in Spannbäumen der Tiefensuche

- *Baumkanten* (tree arcs): Kanten des gewählten Spannbaums
- Kanten, die nicht Baumkanten sind,
  - *Vorwärtskanten* (forward arcs): Kanten, die auf bereits besuchte Nachfolger führen
  - *Rückwärtskanten* (backward arcs): Kanten, die auf bereits besuchte Vorgänger in Richtung der Wurzel führen
  - *Querkanten* (cross arcs): Kanten, die weder auf Vorgänger noch auf Nachfolger führen
- Klassifizierung bei *erstem* Kantendurchlauf von (v, w)

© R.-R. Grigat 2003-2019

- *Baumkante*, wenn $Präorder(w) = 0$
- *Vorwärtskante*, wenn $Präorder(v) < Präorder(w)$
- *Rückwärtskante*, wenn $Präorder(v) \geq Präorder(w) > 0$ und $Postorder(w) = 0$
- *Querkante*, wenn $Präorder(v) \geq Präorder(w) > 0$ und $Postorder(w) \neq 0$

0.14.3.10 Beispiel: Erzeugender Wald aus Tiefensuche

Beispiel

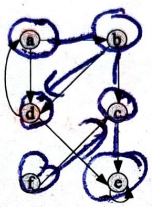
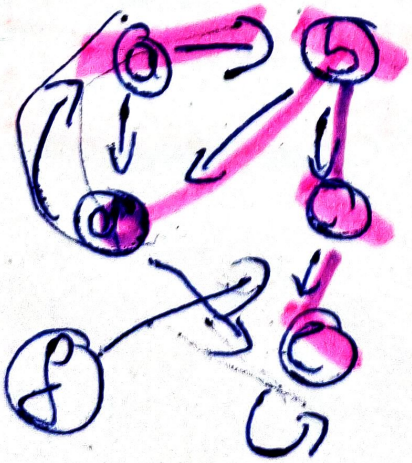


Abbildung 119: Digraph G

Knoten	a	b	c	d	e	f
Präorder	1	2	3	5	4	6
Postorder	5	4	2	3	1	6

Tabelle 35: Prä- und Postorder für Digraph G

- Baumkanten: (a,b), (b,c), (c,e), (b,d)
- Vorwärtskanten: (a,d)
- Rückwärtskanten: (d,a), (e,e)
- Querkanten: (f,c), (d,e)



Prä a b c d e f
1 2 3 5 4 6
1

© R.-R. Grigat 2003-2019

0.14.3.11 Erzeugender Wald aus Breitensuche

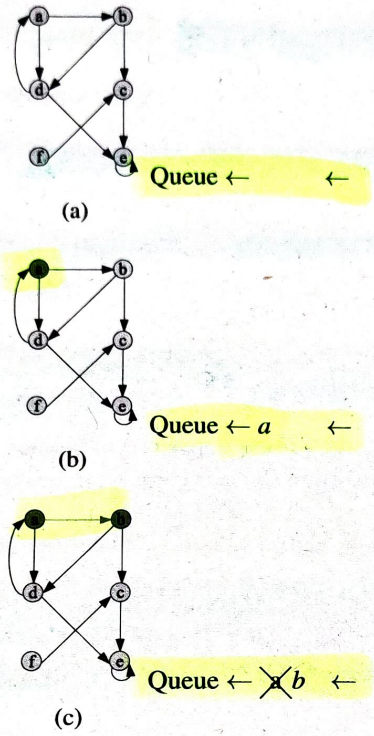
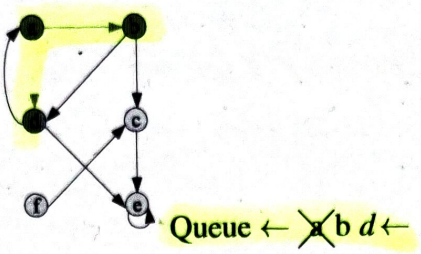
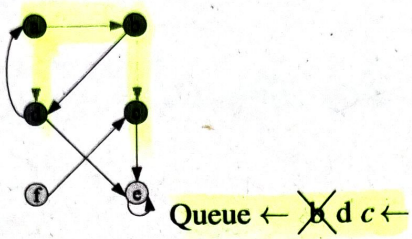


Abbildung 120: Breitensuche

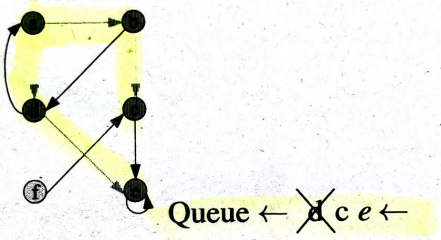
© R.-R. Grigat 2003-2019



(d)



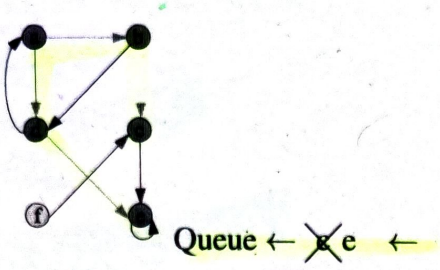
(e)



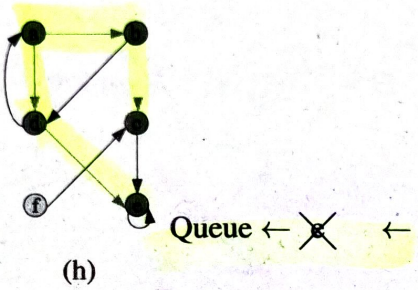
(f)

Abbildung 120: Breitensuche (cont.)

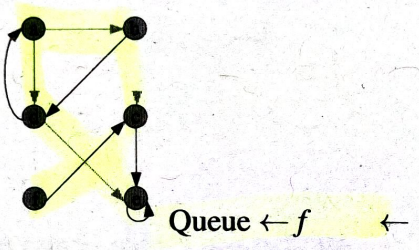
© R.-R. Grigat 2003-2019



(g)

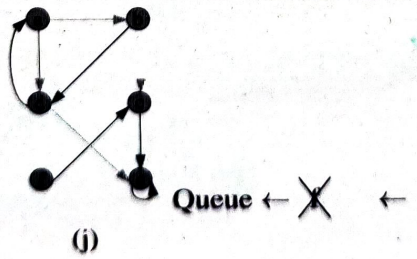


(h)



(i)

Abbildung 120: Breitensuche (cont.)



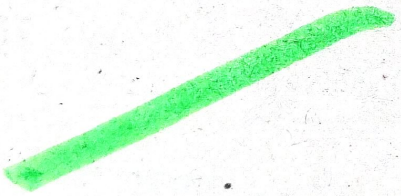
(j)

Abbildung 120: Breitensuche (cont.)

- Suchen des Wurzelknotens a
- Suchen der Nachfolgerknoten von a: b, d
  - Knoten b in die Warteschlange
  - Knoten d in die Warteschlange

© R.-R. Grigat 2003-2019

- Suchen der Nachfolgerknoten von b: c, d
  - Knoten c in die Warteschlange
  - d bereits besucht
- Suchen der Nachfolger von d: a, e
  - a bereits besucht
  - Knoten e in die Warteschlange
- Suchen der Nachfolger von c: e
  - e bereits besucht
- Suchen der Nachfolger von c: e
  - e bereits besucht
- Suchen der Nachfolger von e: keine
- Bei Wurzelknoten f erneut beginnen
- f hat keine unbesuchten Nachfolgerknoten



© R.-R. Grigat 2003-2019

0.14.3.12 Kantentyp bei Breitensuche

- Kante (u,v) von u nach v, Wurzel w
- Tiefen im Spannbaum $t(u)$ und $t(v)$
- Knoten beim ersten Besuch markieren
- Prä- und Postorder hier nicht relevant

Kantentyp (u,v)	Gerichteter Graph
Baumkante	v ist noch unbesucht, $t(v) = t(u)+1$
Vorwärtskante	keine
Rückwärtskante	v ist besucht, liegt auf Pfad $w \to u$, $0 \leq t(v) < t(u)$
Querkante	v ist besucht, liegt *nicht* auf Pfad $w \to u$, $0 < t(v) \leq t(u)+1$

0.14.3.13 Vergleich von Breiten- und Tiefensuche

Beispiel

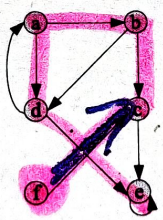


Abbildung 121: Digraph G

- Tiefensuche
  - Baumkanten: (a,b), (b,c), (c,e), (b,d)
  - Vorwärtskanten: (a,d)
  - Rückwärtskanten: (d,a), (e,e)
  - Querkanten: (f,c), (d,e)
- Breitensuche

© R.-R. Grigat 2003-2019

- Baumkanten: (a,b), (a,d), (b,c), (d,e)
- Vorwärtskanten bei Breitensuche nicht definiert
- Rückwärtskanten: (d,a), (e,e)
- Querkanten: (b,d), (c,e), (f,c)

0.14.3.14 Azyklischer Digraph

Definition Dag (directed acyclic graph)
Ein Digraph ist genau dann *azyklisch* (zyklenfrei), wenn er keine Rückwärtskanten enthält.

Beispiel

- durch Entfernung der Kanten (d,a) und (e,e) erhält man einen azyklischen Digraphen

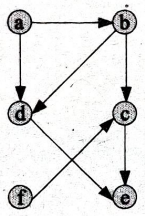


Abbildung 122: Ein azyklischer Digraph

0.14.3.15 Topologisches Sortieren

Definition: Topologische Ordnung für zyklenfreien Digraphen
Kante $(v,w) \in E \Rightarrow Ordnung(v) < Ordnung(w)$

Algorithmus zum topologischen Sortieren

```
while(Knotenmenge nicht leer)
{
  entferne Knoten mit Eingangsgrad 0 aus Knotenmenge;
  füge diesen Knoten ans Ende der sortierten Knotenliste;
}
```

- Der Algorithmus versagt bei Zyklen. Vor topologischem Sortieren sind also Rückwärtskanten zu entfernen.

© R.-R. Grigat 2003-2019

0.14.3.16 Beispiel zu topologischem Sortieren

Beispiel

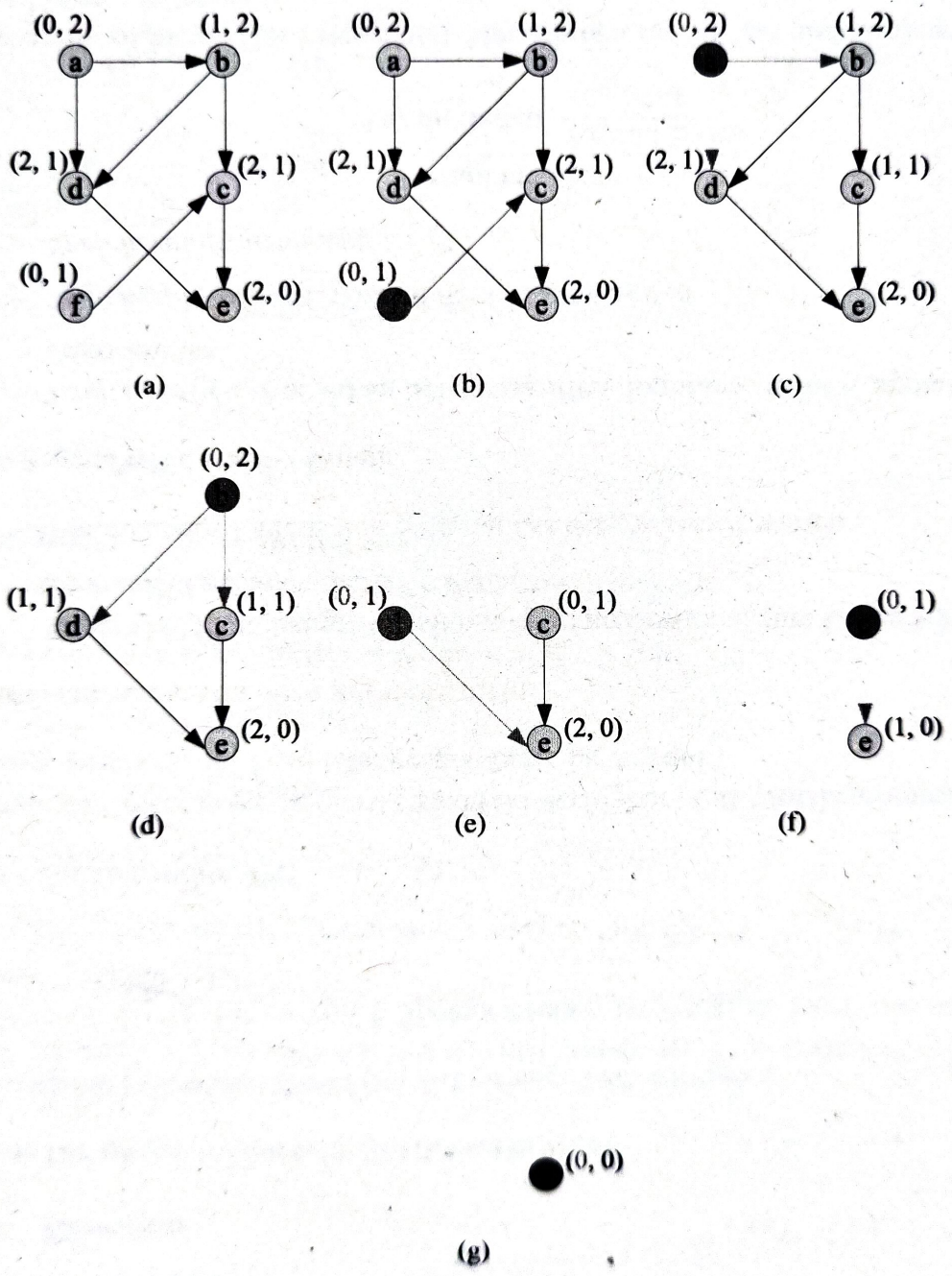


Abbildung 123: Topologisches Sortieren

- Topologische Ordnung: *f, a, b, d, c, e*

© R.-R. Grigat 2003-2019

- Die topologische Ordnung ist jedoch nicht eindeutig

© R.-R. Grigat 2003-2019

0.15 Externes Sortieren

0.15.1 Einleitung

0.15.1.1 Einleitung

Annahme für bisher behandelte Sortierverfahren:

- Sämtliche Daten sind direkt (im Arbeitsspeicher) verfügbar
- Kostenannahme: Lesen von k Speicherstellen ist k-mal so teuer wie das Lesen einer Speicherstelle

Annahme für folgenden Teil

- Daten sind nicht notwendigerweise direkt verfügbar (z. B.: Auslagerungsdatei des Hauptspeichers, auf Festplatte gespeicherte Datenbank)
- Kostenannahme von oben gilt nicht mehr:
  - Kosten bei Plattenzugriffen sind in etwa proportional zum Laden von ganzen Daten*blöcken* (auch *Seiten* genannt)
  - Während des Ladens von Seiten muss der Prozessor warten
- Kostenmaße für *externe Daten*:
  - *Laufzeitkosten*: Anzahl an Seitenzugriffen dominiert, nicht Komplexität des Algorithmus
  - *Speicherplatzkosten*: Anzahl der belegten Seiten
  - *Speicherplatzausnutzung*:

$$\frac{\text{Anzahl benutzter Bytes}}{\text{Anzahl Seiten} \cdot \frac{\text{Anzahl Bytes}}{\text{Seite}}}$$

$\Rightarrow$ Spezielle effiziente Datentypen und Algorithmen für Suchen und Sortieren externer Daten notwendig

$\Rightarrow$ Effizienz bedeutet möglichst wenig Seitenzugriffe

$\Rightarrow$ externe Variante des Datentyps Dictionary gesucht

© R.-R. Grigat 2003-2019

0.15.2 Vielweg-Suchbäume und B-Bäume

0.15.2.1 Datenstruktur für externe Daten

Idee:

- Baum mit *Speicherseiten als Knoten eines Suchbaums*
- Effizienzziel: *möglichst wenig Seitenzugriffe* $\Longrightarrow$ Forderung: *möglichst geringe Pfadlänge*
- Optimal sind breite Suchbäume (großer Knotengrad d) wegen ihrer geringen Höhe $h = \lceil \log_d(n \cdot (d-1)+1) \rceil - 1 \leq \lceil \log_d n \rceil$
- Anzahl an Werten pro Knoten ist begrenzt durch die Größe einer Speicherseite
- *Vielweg-Suchbaum* (allgemeiner Suchbaum) vom *Grad d*:
  - sortierter Baum
  - maximal d Werte pro Knoten
  - maximal $d+1$ Söhne
  - Erweiterung der binären Suchbäume

0.15.2.2 Vielweg-Suchbäume (allgemeine Suchbäume)

Definition

- Leerer Vielweg-Suchbaum hat Schlüsselmenge $\{\}$
- Seien $T_0, \ldots, T_s$ Vielweg-Suchbäume, mit Schlüsselmengen $\overline{T}_0, \ldots, \overline{T}_s$
- Sei $< k_1, \ldots, k_s >$ Folge von Schlüsseln mit $k_1 < k_2 < \ldots < k_s$
- Die Folge $< T_0, k_1, T_1, k_2, T_2, k_3, \ldots, k_s, T_s >$ ist genau dann Vielweg-Suchbaum, wenn gilt:

$$\begin{array}{ll} \forall x \in \overline{T}_i: & k_i < x < k_{i+1} \quad \text{für } i = 1, \ldots, s-1 \\ \forall x \in \overline{T}_0: & x < k_1 \\ \forall x \in \overline{T}_s: & k_s < x \end{array}$$

Seine Schlüsselmenge ist $\{k_1, \ldots, k_s\} \cup \bigcup_{i=0}^{s} \overline{T}_i$

© R.-R. Grigat 2003-2019

Beispiel

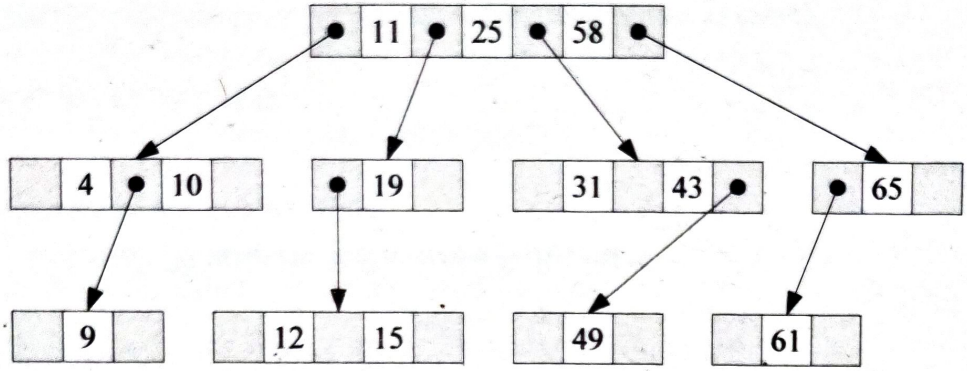


Abbildung 124: Vielweg-Suchbaum

0.15.2.3 B-Bäume

Definition: B-Baum der Ordnung m
Ein B-Baum ist ein Vielweg-Suchbaum, für den zusätzlich gilt:

- Anzahl der Schlüssel s in jedem Knoten: $m \leq s \leq 2m$
  - Ausnahme Wurzelknoten: $1 \leq s \leq 2m$
- Blätter besitzen nur Schlüssel
- Jeder *innere* Knoten mit s Schlüsseln hat $s+1$ Söhne
- Alle Pfadlängen von der Wurzel zu einem Blatt sind gleich
  - innere Knoten dürfen keine leeren Teilbäume besitzen
  - Ausnahme: Wurzelknoten mit genau 1 Schlüssel kann 0 Blätter, 2 Blätter oder 2 Teilbäume besitzen

© R.-R. Grigat 2003-2019

Beispiel

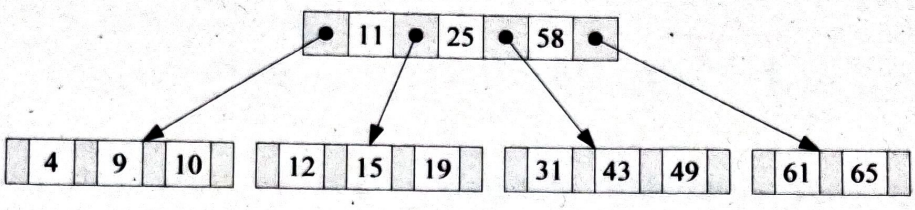


Abbildung 125: B-Baum

0.15.2.4 Minimale Schlüssel- und Knotenzahl in B-Baum

Minimal gefüllter B-Baum der Höhe h, Ordnung m

- Wurzelknoten besitzt zwei Teilbäume T_L und T_R
- Min. Anzahl an Knoten und Schlüsseln:

$$\begin{aligned}
\#\text{Knoten}_{\min}(T_L) &= \sum_{i=0}^{h-1}(m+1)^i = \frac{(m+1)^h-1}{(m+1)-1} \\
&= \frac{(m+1)^h-1}{m} \\
\#\text{Schlüssel}_{\min}(T_L) &= m\cdot\frac{(m+1)^h-1}{m} \\
&= (m+1)^h-1 \\
\#\text{Schlüssel}_{\min}(T) &= 2\cdot(m+1)^h-1 \qquad (23)
\end{aligned}$$

$\Rightarrow$ In einem Baum der Höhe h sind mindestens $2\cdot(m+1)^h-1$ Schlüssel vorhanden

0.15.2.5 Maximale Höhe eines B-Baumes

Maximale Höhe eines B-Baumes der Ordnung m

- Es seien n Schlüssel in einem B-Baum der Höhe h gespeichert.

© R.-R. Grigat 2003-2019

- Dann gilt mit Gleichung 23:

$$\begin{aligned} n &\geq 2 \cdot (m+1)^h - 1 \\ (m+1)^h &\leq \frac{n+1}{2} \\ h &\leq \log_{(m+1)} \left(\frac{n+1}{2} \right) \\ &\in O(\log n) \end{aligned}$$

⇒ Bei n Schlüsseln muss die Höhe des Baumes kleiner sein als $\log_{(m+1)} \left(\frac{n+1}{2} \right)$, ansonsten wäre er geringer als minimal besetzt

0.15.2.6 Maximale Schlüssel- und Knotenzahl in B-Baum

Maximal gefüllter B-Baum der Höhe h, Ordnung m

$$\begin{aligned} \#\text{Knoten}_{\max}(T) &= \sum_{i=0}^{h} (2m+1)^i = \frac{(2m+1)^{h+1} - 1}{(2m+1) - 1} \\ &= \frac{(2m+1)^{h+1} - 1}{2m} \\ \#\text{Schlüssel}_{\max}(T) &= 2m \cdot \frac{(2m+1)^{h+1} - 1}{2m} \\ &= (2m+1)^{h+1} - 1 \end{aligned} \tag{24}$$

⇒ In einem Baum der Höhe h sind maximal $(2m+1)^{h+1} - 1$ Schlüssel vorhanden

0.15.2.7 Beispiel: Maximale Höhe eines B-Baumes

Beispiel

- Seiten der Größe 2 kB
- Größe für Schlüssel und Zeiger zu weiteren Knoten: 4 Bytes

⇒ $(2m + (2m+1)) \cdot 4 \leq 2048 \Leftrightarrow m = 127$

- Es sollen $n = 2 \cdot 10^6$ Daten gespeichert werden.

© R.-R. Grigat 2003-2019

- Maximale Höhe des B-Baumes mit $h \leq \log_{(m+1)}\left(\frac{n+1}{2}\right)$: $h_{max} = \lfloor \log_{128} \frac{2 \cdot 10^6 + 1}{2} \rfloor = 2$
- Probe:
  - Minimale Anzahl an Schlüsseln mit Gleichung (23) ist
    $$2 \cdot (m+1)^h - 1 = 2 \cdot 128^2 - 1 = 32767$$
  - Maximale Schlüsselzahl ergibt sich nach (24)
    $$(2m+1)^{h+1} - 1 = 255^3 - 1 = 16581374$$

0.15.2.8 Algorithmen auf B-Bäumen

Grundoperationen

- *member*: Suchen eines Elements
- *insert*: Einfügen eines Elements
- *delete*: Löschen eines Elements
- Behandlung von Überlauf und Unterlauf eines Knotens
  - *split*: Teilen von Knoten
  - *balance*: Länge der Knoten innerhalb einer Ebene angleichen
  - *merge*: Verschmelzen von Knoten

0.15.2.9 Suchen in B-Bäumen

Die Funktion *member*

- Binäre Suchbäume liefern geeigneten Algorithmus für Suche in B-Bäumen.
- Innerhalb eines Knotens kann linear gesucht werden, also mit $O(m)$.
- CPU-Zeit gesamt $O(m \cdot h) = O(m \log_{m+1} n) = O(\log n)$

© R.-R. Grigat 2003-2019

0.15.2.10 Einfügen in B-Bäumen

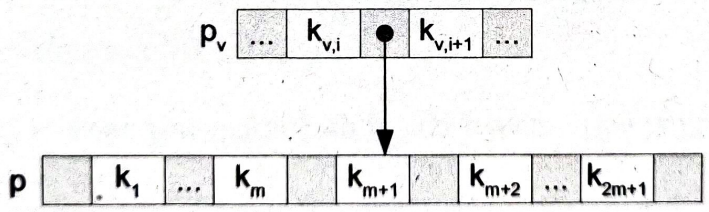
Die Funktion *insert*
Füge Element e in Baum ein anhand Schlüssel $s_e = \text{key}(e)$

- Suche e im Baum beginnend bei Wurzel
- Falls e nicht gefunden wurde:
  - p sei das Blatt, in dem die Suche endete
  - Füge e in das Blatt p an der richtigen Position ein
- Falls #Schlüssel$(p) = 2m + 1$:
  - Überlauf (*Overflow*)
  - Zerteile p in zwei Knoten mit gültiger Länge (*split*)

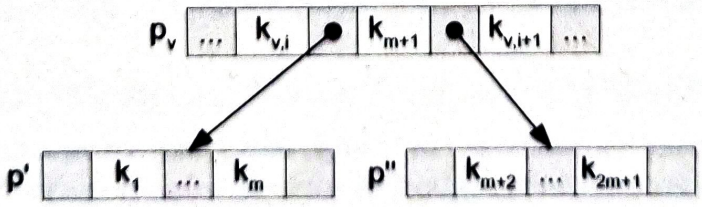
0.15.2.11 Überlauf beim Einfügen in B-Bäumen

Die Funktion *split*

- Fall 1: p hat Vater p_v:



(a)



(b)

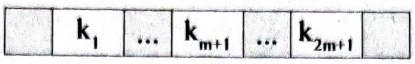
Abbildung 126: *split*, falls p nicht Wurzel

© R.-R. Grigat 2003-2019

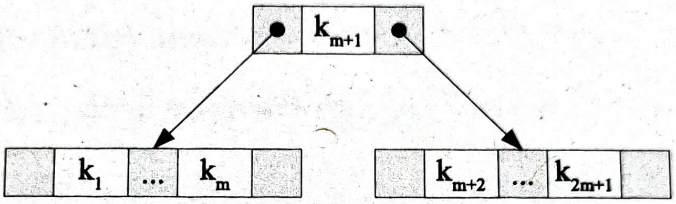
- Prüfe anschließend p_v auf Überlauf

$\Rightarrow$ Überlauf kann sich von einem Blatt bis zur Wurzel fortsetzen

- Fall 2: p ist Wurzelknoten: Baum wächst "nach oben"



(a)



(b)

Abbildung 127: *split* mit neuer Ebene, falls p Wurzel

0.15.2.12 Löschen von Elementen in B-Bäumen

Die Funktion *delete*
Lösche existierendes Element e aus Baum

- *Suche e im Baum beginnend bei Wurzel*
- *Falls e auf einem Knoten gefunden wurde, der kein Blatt ist:*
  - Suche nächstgrößeren Wert e', der auf einem Blatt liegt (*äquivalenter Wert*).
  - Vertausche e mit e'.
- *Lösche e vom Blatt.*
- Falls das Blatt danach $m-1$ Schlüssel enthält und *keine* Wurzel ist:
  - Unterlauf (*Underflow*)

© R.-R. Grigat 2003-2019

- *balance*: Falls Nachbarknoten mit überzähligen Schlüsseln $s' > m$ existiert, so übernimm Schlüssel von dort
- *merge*: Falls nein, dann existiert Nachbarknoten mit Minimalzahl $s' = m$ an Schlüsseln, da Blatt keine Wurzel ist. Verschmelze beide Knoten.

0.15.2.13 Balance-Operation in B-Bäumen

Die Funktion *balance*

- Durchführbar, wenn Nachbarknoten p' von p mehr als m Schlüssel hat, d.h. $s > m$

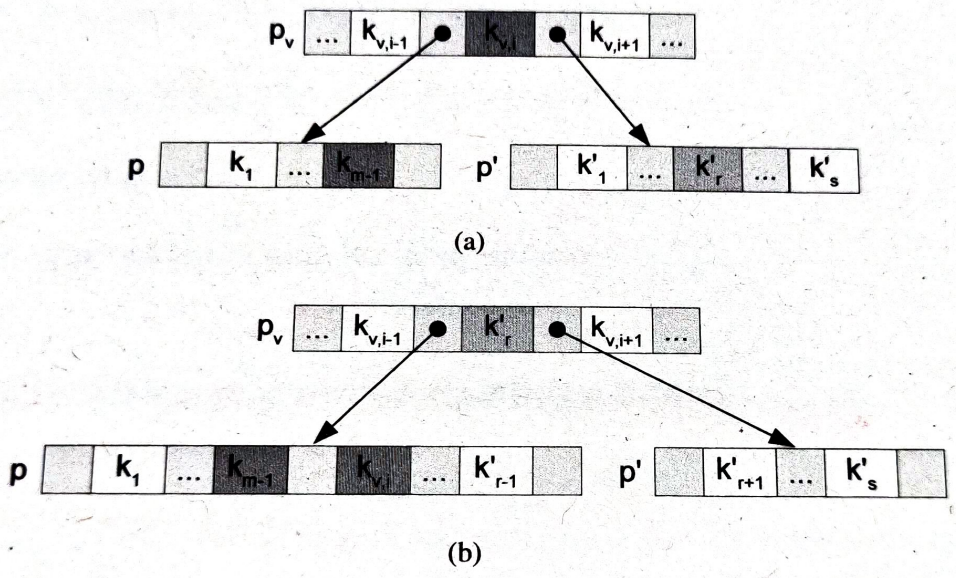


Abbildung 128: Balance-Operation in B-Bäumen

- Teilt Folge $\langle k_1, k_2, \ldots, k_{m-1}, k_{v,i}, k'_1, k'_2, \ldots, k'_s \rangle$ mittig auf
- Schnitt liegt in Nachbarknoten p' bei Knoten k'_r mit Index

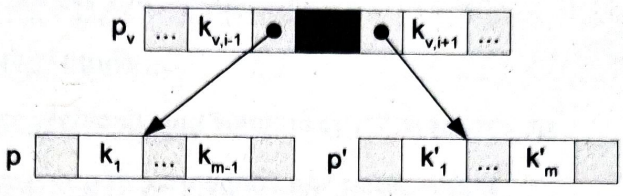
$$r = \left\lceil \frac{m+s}{2} \right\rceil - m$$

© R.-R. Grigat 2003-2019

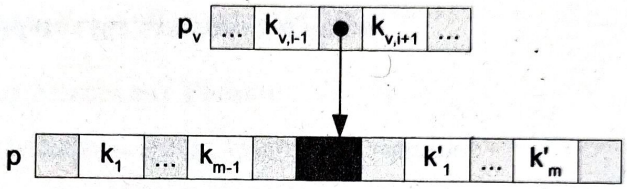
0.15.2.14 Verschmelzen von B-Bäumen

Die Funktion *merge*

- Durchführbar, wenn Nachbarknoten p' von p genau m Schlüssel hat



(a)



(b)

Abbildung 129: Verschmelzen von B-Bäumen

- Anschließend: Vater p_v auf Unterlauf testen und Unterlauf ggf. beheben
- Falls Unterlauf von p_v eintritt und p_v Wurzel war:
  - Neue Wurzel ist der durch Merge-Operation entstandene Knoten p

0.15.2.15 Komplexitätsanalyse von B-Bäumen

Komplexität

- Platzbedarf: $O(n)$
- Speicherplatzausnutzung: Garantiert größer als 50%
- Nur $O(h) = O(\log_{m+1} n)$ Plattenoperationen
- Pro Knoten $O(m)$ Schritte
- Komplexität der vorgestellten Algorithmen: $O(m \log_{m+1} n) = O(\log n)$

© R.-R. Grigat 2003-2019

0.15.3 Algorithmen für externes Sortieren

Annahmen

- Gegeben seien n Datensätze
- Datensätze extern sequenziell als *Seitenfolge* gespeichert
  - Magnetband
  - Festplatte
- Für n Datensätze werden s Seiten benötigt
- Jede Seite enthält *ausschließlich* d_s Datensätze
  - keine Zeiger auf Nachfolgeknoten o. ä.
  - bessere Ausnutzung des Speicherplatzes
  - keine B-Baum-Struktur

Ziel

- Effizientes Sortieren der Datensätze
- Effizient:
  - *Sequenzielle Verarbeitung*, um unnötiges Warten zu vermeiden (Schreib-Lesekopf-Bewegungen, Spulen von Magnetbändern).
  - Sequenzielle Verarbeitung impliziert wenig Seitenzugriffe!
- Mindestens 4 Geräte bzw. Dateien (Festplatten, Bandmaschinen) benötigt
  - 2 Geräte f_1 und f_2 zum Einlesen von Daten
  - 2 Geräte g_1 und g_2 zum Schreiben von Daten

© R.-R. Grigat 2003-2019

0.15.3.1 Sortierprozedur für externes Sortieren

Sortieralgorithmus (externes Mergesort)

1. *Aufteilungsschritt*:
   - Daten auf zwei sequenzielle Eingabedateien f_1 und f_2 aufteilen, z. B. auf zwei Bandmaschinen (Teilfolgenlänge 1)
2. *Sortierschritt*:
   - Je eine Teilfolge von f_1 und f_2 ähnlich Mergesort zu einer Resultatfolge doppelter Länge sortiert zusammenfassen
   - Resultatfolgen folgenweise alternierend in eine der Ausgabedateien g_1 und g_2 schreiben
   - Beide vorherigen Schritte wiederholen bis f_1 und f_2 leer
3. *Tauschschritt*: Tausche g_1, g_2 mit f_1, f_2
4. Solange f_1 und f_2 noch je zwei oder mehr Resultatfolgen enthalten weiter bei Sortierschritt
5. *Verschmelzschritt*:
   - Fasse Datensätze f_1 und f_2 sortiert zusammen

0.15.3.2 Nomenklatur für externes Sortieren

Definition

- *Phase:* Sortierschritt in Verbindung mit Tauschschritt
  - Externes Sortieren läuft somit in $O(\log n)$ Phasen ab.
- *Lauf:* sortierte Teilfolge
  - Zu Beginn des Sortierverfahrens:
    - Daten unsortiert
    - n Läufe der Länge 1
  - In jeder Phase verdoppelt sich Länge der in g_1, g_2 abgelegten Läufe und halbiert sich Zahl der Läufe.
  - Am Ende des Sortierverfahrens:

© R.-R. Grigat 2003-2019

- Daten sortiert
- Nur ein verbleibender Lauf der Länge n

0.15.3.3 Beispiel: Externes Mergesort

Beispiel
Aufteilungsschritt: Die zu sortierende Sequenz wird auf zwei Geräte bzw. Dateien f_1 und f_2 aufgeteilt

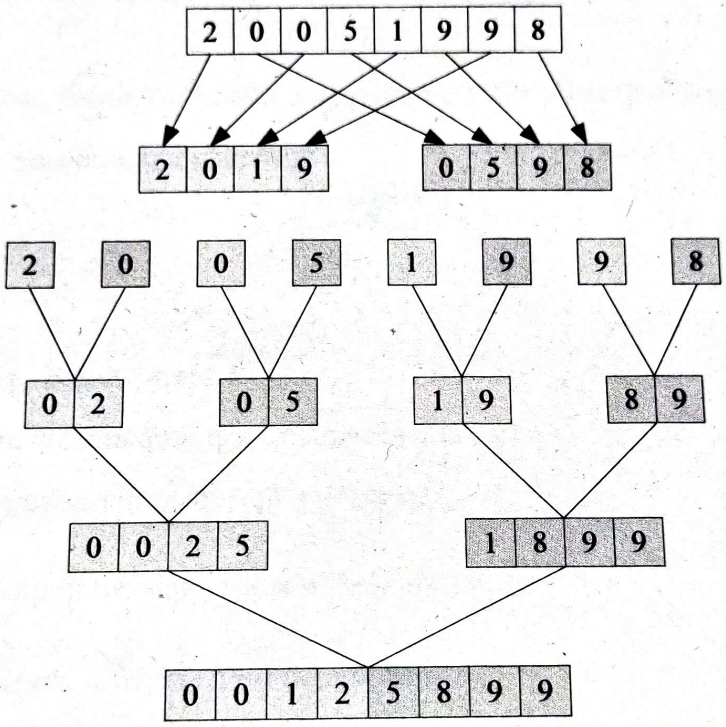


Abbildung 130: Externes Sortieren

Phase 1
- 2 Läufe der Länge 1 werden aus f_1 und f_2 gelesen
- Erster sortierter Lauf der Länge 2 wird in g_1 gespeichert
- Weitere Läufe werden alternierend in g_2 und g_1 gespeichert

Phase 2
- Tausch der Geräte bzw. Dateien $f_1 \leftrightarrows g_1, f_2 \leftrightarrows g_2$
- Läufe der Länge 2 werden aus f_1 und f_2 gelesen...
- ...und nach Mergesort alternierend in g_1 und g_2 gespeichert

Phase 3
- Sortierter Lauf der Länge n steht in den letzten 2 Ausgabedateien

© R.-R. Grigat 2003-2019

0.15.3.4 Komplexitätsanalyse

Komplexität von externem Sortieren, n Datensätze, d_s Datensätze pro Seite

- Kosten jeder Phase:
  - Zugriff auf alle Speicherseiten
  - $C_{\text{Kosten pro Phase}}(n, d_s) \in O(\frac{n}{d_s})$
- Anzahl an Phasen: $\lceil \log_2(n) \rceil$

$\Rightarrow$ Gesamtkomplexität: $T(n) \in O(\frac{n}{d_s} \log n) = O(n \log n)$

0.15.3.5 Varianten des Aufteilungsschritts

Verringerung der Anzahl der Phasen

- Bisher: $\lceil \log_2(n) \rceil$ Phasen benötigt
- *Direktes Mischen*
  - Gesamten verfügbaren Hauptspeicher mit m Speicherstellen zum internen Vorsortieren der Anfangsläufe ausnutzen
  - Anfangsläufe *maximaler Länge* $m > 1$ verwenden
- *Vielweg-Mischen*
  - Mehr als zwei Eingabe-/Ausgabedateien $f_1, f_2, \ldots$ und $g_1, g_2, \ldots$ verarbeiten

0.15.3.6 Direktes Mischen

Idee

- Vorsortieren mit Länge m reduziert die Anzahl an Phasen von $\lceil \log_2(n) \rceil$ auf $\lceil \log_2(n/m) \rceil$
- Grenzfall $m = n$: Daten können komplett intern sortiert werden
- Limitierender Faktor: Hauptspeicherkapazität

Beispiel

- 1000 Datensätze sollen sortiert werden ($n = 1000$)

© R.-R. Grigat 2003-2019

- In den Hauptspeicher passen 100 Datensätze
  - beim Aufteilen der Datensätze auf f_1 und f_2 10 Läufe (der Länge 100) erzeugen
- Anschließend noch $\lceil \log_2(1000/100) \rceil = 4$ Phasen durchlaufen

0.15.3.7 Vielweg-Mischen

Idee

- Verwendung von d Ein-/Ausgabedateien: Länge der Läufe erhöht sich um Faktor d pro Phase
- Anzahl an Phasen reduziert sich von $\lceil \log_2(n) \rceil$ auf $\lceil \log_d(n) \rceil$
- Grenzfall $d = n$: Daten können komplett intern sortiert werden
- Limitierender Faktor
  - Hauptspeicherkapazität
  - maximal zulässige Anzahl an gleichzeitig geöffneten Dateien

0.15.3.8 Externes Sortieren: Zusammenfassung

- Die Komplexität bezieht sich bei externen Daten auf die Anzahl an externen Speicher(seiten)-Zugriffen
- Dictionary, das bei Operationen auf möglichst wenig Seitenzugriffe ausgelegt ist: B-Bäume
- Beim Entwurf geeigneter B-Bäume mit möglichst wenig Knoten ist maßgeblich:
  1. Die Seitengröße des Speichers
  2. Der Speicherplatzbedarf der *Zeiger* auf weitere Speicherseiten
  3. Der Speicherplatzbedarf eines Schlüssels
- Sortierverfahren, das auf sequenziellen Datenzugriff ausgelegt ist: Externes Mergesort
- Je nach dem Grad an bereits sortierten Daten ist es sinnvoll, sie als Läufe fester oder variabler Länge vorzubereiten

© R.-R. Grigat 2003-2019

0.16 Dynamische Programmierung

0.16.1 Einleitung

0.16.1.2 Literatur zu dynamischer Programmierung

- Cormen, Leiserson, Rivest, Stein: „Algorithmen - Eine Einführung“ :
  Detaillierte formale Beschreibung, Grundlage für Vorlesung
- Sedgewick, Robert: „Algorithmen in C“ :
  Anschauliche Beispiele

0.16.1.3 Entwicklung eines dynamischen Programms

Zur Erinnerung: Entwicklung eines dynamischen Programms erfolgt in *4 Schritten*:

1. Charakterisierung der *Struktur* einer optimalen Lösung (z. B. als Graph)
2. *Optimale Teilstruktur:* Rekursive Formulierung des Algorithmus auf Basis bereits optimal gelöster Teilprobleme (z. B. für kürzeste Wege im Graphen)
3. Iterative Berechnung bottom-up des *Wertes* aller optimalen Teillösungen (z. B. kürzeste Pfadlänge)
4. Konstruktion der zugehörigen optimalen Lösung aus den berechneten Daten (z. B. Pfad mit kürzester Länge)

Anmerkung

- Für manche Probleme wird nur der *Wert* einer optimalen Lösung benötigt, Schritt 4 kann dann entfallen

© R.-R. Grigat 2003-2019

0.16.2 Ablaufkoordination von Montagebändern

0.16.2.1 Problem: Ablaufsteuerung von Montagebändern in Autofabrik

Aufgabe

- Fabrik fertigt Autos durch Montage von Teilen an Chassis
- Fabrik hat 2 Montagebänder mit n Stationen $S_{i,j}$, Band $1 \leq i \leq 2$, Station $1 \leq j \leq n$
- Station $S_{1,j}$ hat dieselbe Funktion wie $S_{2,j}$, benötigt aber unterschiedliche Zeiten $a_{i,j}$ (z. B. durch Modernisierung)
- Zeit für Übergang von $S_{i,j}$ nach $S_{i,j+1}$ vernachlässigbar
- Zeit für Übergang von Station $S_{i,j}$ zur folgenden Station $j+1$ des anderen Bandes sei $t_{i,j}$
- Zeit, die Chassis benötigt, um zu Band i zu gelangen: e_i
- Zeit, die fertiges Auto benötigt, um Werk von Band i zu verlassen: x_i

Fragestellung
Welcher Pfad muss gewählt werden, um die Fertigungszeit zu minimieren?

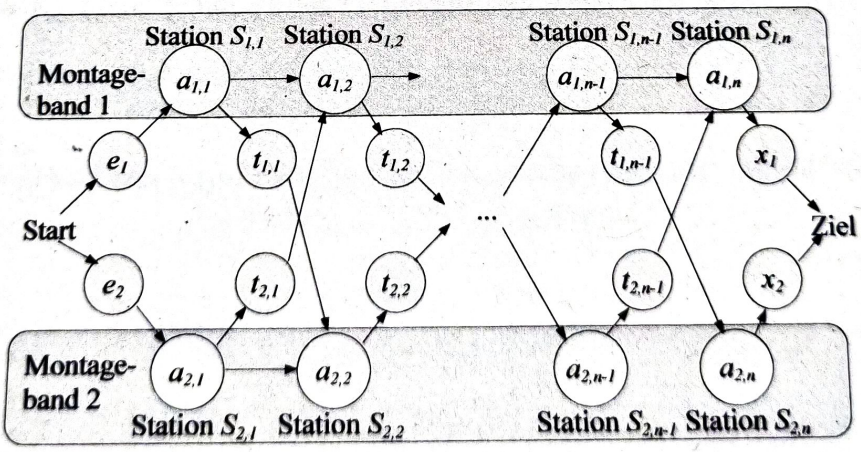


Abbildung 131: Montagebänder

- Komplexität, um für einen gegebenen Pfad Gesamtzeit zu berechnen: $\Theta(n)$
- Anzahl möglicher Pfade: $\Theta(2^n)$

© R.-R. Grigat 2003-2019

- Betrachte: Menge der von Band 1 verwendeten Stationen ist eine Teilmenge aus $\{1,\ldots,n\}$
- Es gibt 2^n solche Teilmengen

$\Rightarrow$ Vollständige Suche für große n nicht durchführbar

$\Rightarrow$ Dynamische Programmierung verwenden

0.16.2.2 Schritt 1/4: Struktur der optimalen Lösung

Schnellstmöglicher Pfad vom Start zu $S_{1,j}$

- Für $j = 1$ gibt es nur einen möglichen Pfad
- Für $2 \leq j \leq n$ gibt es 2 Möglichkeiten:
  - Chassis könnte von $S_{1,j-1}$ gekommen sein
    - Übergang benötigt keine Zeit
  - Chassis könnte von $S_{2,j-1}$ gekommen sein
    - Übergang benötigt Zeit $t_{2,j-1}$
- Beobachtung: Bestimmung der optimalen Lösung des Problems (schnellster Pfad zu $S_{1,j}$) beinhaltet optimale Lösungen zu Teilproblemen (Bestimmung des schnellsten Pfades zu $S_{1,j-1}$ und $S_{2,j-1}$)
  - Diese Eigenschaft heißt *optimale Teilstruktur*
  - *Optimale Teilstruktur* und *überlappende Teilprobleme* sind die zwei wichtigsten Kriterien für Anwendbarkeit dynamischer Programmierung

0.16.2.3 Schritt 2/4: Rekursive Formulierung der minimalen Kosten

Rekursive Beschreibung der Kosten des schnellstmöglichen Pfades

- Sei $f_i[j]$ schnellstmögliche Zeit vom Start zu $S_{i,j}$

© R.-R. Grigat 2003-2019

- Sei f^* kürzeste Zeit von Start zu Ziel

Initialisierung:
$$f_1[1] = e_1 + a_{1,1} \quad (25)$$
$$f_2[1] = e_2 + a_{2,1} \quad (26)$$
Induktion:
$$f_1[j] = \min(f_1[j-1] + a_{1,j}, \quad (27)$$
$$f_2[j-1] + t_{2,j-1} + a_{1,j}) \quad (28)$$
$$f_2[j] = \min(f_2[j-1] + a_{2,j}, \quad (29)$$
$$f_1[j-1] + t_{1,j-1} + a_{2,j}) \quad (30)$$
$$\forall\, 2 \leq j \leq n$$
Terminierung:
$$f^* = \min(f_1[n] + x_1, f_2[n] + x_2) \quad (31)$$

0.16.2.4 Schritt 3/4: Berechnung der minimalen Kosten

Ungünstig: **Rekursive Berechnung, also keine Verwendung optimaler Teilstruktur**

- Komplexität des rekursiven Auswertens der Rekursionsgleichungen (25) bis (31): $\Theta(2^n)$
- Begründung: Sei $r_i(j)$ die Anzahl, wie oft bei Berechnung von f^* auf $f_i[j]$ zurückgegriffen wird.

$$r_1(n) = r_2(n) = 1 \quad (32)$$
$$r_1(j) = r_2(j) = r_1(j+1) + r_2(j+1)\ \forall\, 1 \leq j < n \quad (33)$$

$\Rightarrow$ $r_i(j) = 2^{n-j}$, z. B. $f_1[1]$ ist 2^{n-1} mal zu berechnen

Besser: **Iterative Berechnung der optimalen Teilstrukturen**

- Für $j \geq 2$ hängt Wert $f_i[j]$ nur von $f_1[j-1]$ und $f_2[j-1]$ ab.
- Also: Werte $f_i[j]$ in Reihenfolge steigender Stationszahlen j berechnen und in Tabelle speichern

© R.-R. Grigat 2003-2019

0.16.3.3 Worterkennung

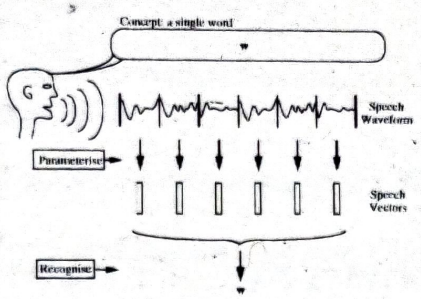


Abbildung 134: Worterkennung

Annahmen

- Jedes Sprachsignal entspricht genau einem Symbol (Wort)
- Alle Wörter entstammen einem gegeben Wörterbuch
- Jedes gesprochene Wort wird durch eine Beobachtungssequenz $\mathbf{O} = \mathbf{o}_1, \mathbf{o}_2, \ldots, \mathbf{o}_T$ repräsentiert, wobei $\mathbf{o}_t$ der zum Zeitpunkt t beobachtete Sprachvektor ist

0.16.3.4 Hidden Markov Modell

Definition Markov Modell

- Ein Markov Modell M ist ein Zustandsautomat
  - Zu jedem ganzzahligen Zeitpunkt t im Intervall $0 \leq t \leq T+1$ findet ein Zustandsübergang statt
  - Der Übergang vom Zustand i zum Zustand j hat die Wahrscheinlichkeit a_{ij} und es gilt $\sum_j a_{ij} = 1$
  - Zu jedem Zeitpunkt t, wenn ein Zustand j erreicht wird, wird ein Beobachtungsvektor $\mathbf{o}_t$ gemäß der (Amplituden-)Wahrscheinlichkeitsdichte $b_j(\mathbf{o}_t)$ erzeugt
    - Ausnahme: Eingangszustand $x(0)$ and Ausgangszustand $x(T+1)$ erzeugen keine Beobachtungsvektoren. Dies erleichtert Hintereinanderschaltung von Markov Modellen.
- Nur die Beobachtungssequenz $\mathbf{O}$ ist bekannt, Zustandssequenz X ist verdeckt (hidden) $\Longrightarrow$ *Hidden Markov Model* (HMM)

© R.-R. Grigat 2003-2019

- Ein *Hidden Markov Model M* wird durch folgende Parameter charakterisiert:
  - Anzahl der Zustände N_M
  - Übergangswahrscheinlichkeiten a_{ij}
  - Wahrscheinlichkeitsdichten $b_j(\mathbf{o}_t)$ für Amplituden der Beobachtungsvektoren $\mathbf{o}_t$ bei Erreichen des Zustandes j

0.16.3.5 Beispiel: Markov Modell *M*

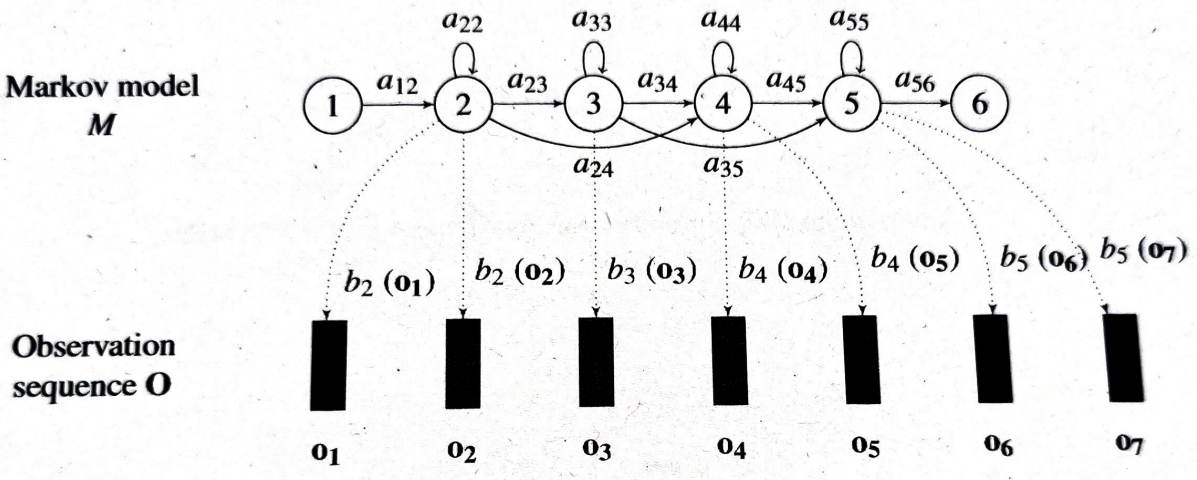


Abbildung 135: Beispiel eines (nicht verdeckten) Markov Models

- Anzahl N_M der Zustände des Modells M ist $N_M = 6$
- Länge N_T der Beobachtungssequenz $\mathbf{O}$ bzw. Anzahl der Beobachtungen $\mathbf{o}_t$ ist $N_T = 7$
- Zustandssequenz X ist gesucht, Lösung hier $X = \langle 1,2,2,3,4,4,5,5,6 \rangle$
- Zustandssequenz X: $x(0) = 1$ (Eingang, keine Beobachtung), $x(1) = 2, x(2) = 2, \ldots, x(T) = 5, x(T+1) = 6$ (Ausgang, keine Beobachtung)
- Verbundwahrscheinlichkeitsdichte, dass Beobachtungssequenz $\mathbf{O}$ aus Zustandssequenz X des Modells M erzeugt wurde:

$$P(\mathbf{O}, X|M) = a_{12} b_2(\mathbf{o}_1) a_{22} b_2(\mathbf{o}_2) a_{23} b_3(\mathbf{o}_3) \ldots \tag{34}$$

© R.-R. Grigat 2003-2019

0.16.3.6 *Maximum likelihood*-Klassifikation

Worterkennung

- Wahrscheinlichkeitsdichte $P(\mathbf{O}|M)$ wird für Klassifikation benötigt
- Berechnung mit Summe über alle möglichen Zustandssequenzen: $X = \langle x(0), x(1), x(2), \ldots x(T), x(T+1) \rangle$:

$$P(\mathbf{O}|M) = \sum_X a_{x(0)x(1)} \prod_{t=1}^{T} b_{x(t)}(\mathbf{o}_t) a_{x(t)x(t+1)} \tag{35}$$

  mit Eingangszustand $x(0)$, Ausgangszustand $x(T+1)$

- Approximation von $P(\mathbf{O}|M)$ durch wahrscheinlichste Zustandssequenz X:

$$\hat{P}(\mathbf{O}|M) = \max_X \{ a_{x(0)x(1)} \prod_{t=1}^{T} b_{x(t)}(\mathbf{o}_t) a_{x(t)x(t+1)} \} \tag{36}$$

Anmerkungen

- Wenn Beobachtung $\mathbf{o}_t$ unwahrscheinlich ist (suboptimale Zustandssequenz X), dann sind $b_{x(t)}(\mathbf{o}_t)$ und damit $P(\mathbf{O}|M)$ nahezu 0
- Komplexität der Berechung von $P(\mathbf{O}|M)$ ist $O(N_T {N_M}^{N_T})$
  - N_M mögliche Zustände $\Rightarrow O({N_M}^{N_T})$ mögliche Zustandssequenzen
  - Für jede Zustandssequenz sind $O(N_T)$ Multiplikationen nötig
- Gleichung 36 kann mit *Viterbi-Algorithmus* (dynamische Programmierung) effizient berechnet werden
- Annahme: Parameter $\{a_{ij}\}$ und $b_j(\mathbf{o}_t)$ sind bekannt
  - Parameter der Modelle M_k werden automatisch aus einer gegebenen Trainingsmenge bestimmt
- Resultat der Klassifikation ist das Wort ω_K, dessen Modell M_K die größte Maximalwahrscheinlichkeit aller Worte $\{\omega_k\}$ besitzt

$$\hat{P}(\mathbf{O}|M_K) = \max_k \{ \hat{P}(\mathbf{O}|M_k) \} \tag{37}$$

© R.-R. Grigat 2003-2019

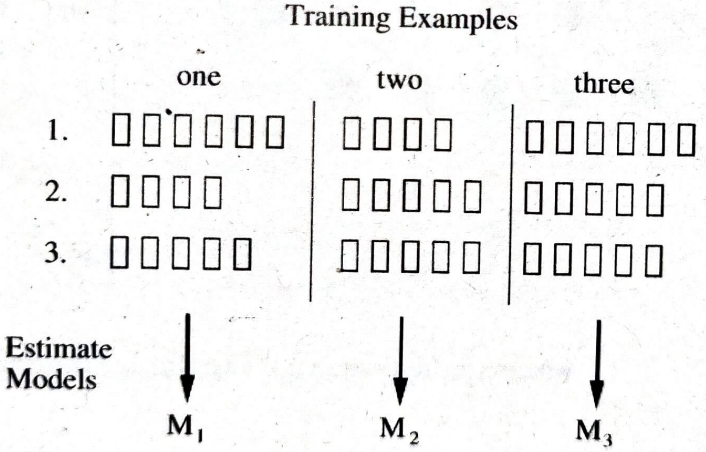
0.16.3.7 Zusammenfassung

Worterkennung

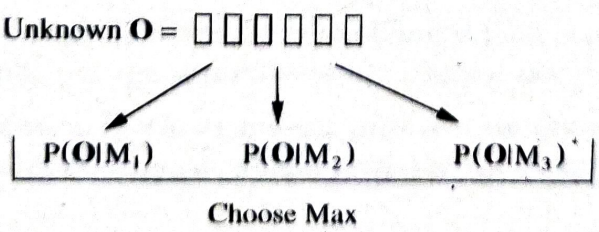
1. Hidden Markov Modelle trainieren
   - Ein Modell M_k pro Wort für jedes Wort ω_k aus dem Wörterbuch
   - Pro Wort werden mehrere Sprachsignale für das Training benötigt
2. Wahrscheinlichkeit, daß ein Modell M_k die Beobachtungssequenz $\mathbf{O}$ erzeugt, wird approximiert mit $\hat{P}(\mathbf{O}|M_k)$
3. Das Modell mit der größten Wahrscheinlichkeit identifiziert das gesprochene Wort

0.16.3.8 Beispiel: Wörterbuch {"one", "two", "three"}

Beispiel



(a) Training der Modelle, 3 Trainingssequenzen pro Wort



(b) Worterkennung: Unbekannte ist das Markov Modell M_i für gegebene Beobachtungssequenz $\mathbf{O}$

© R.-R. Grigat 2003-2019

0.16.3.9 Viterbi-Algorithmus

Optimale Zustandssequenz

- Bestimme $\max_k \{\hat{P}(\mathbf{O}|M_k)\}$

Optimale Teilstruktur der Lösung

- Ermittle die Wahrscheinlichkeiten $\Phi_j(t)$ aller Zustände j des Modells M zum Zeitpunkt t nur unter Verwendung von
  - der aktuellen Beobachtung $\mathbf{o}_t$ und
  - den Wahrscheinlichkeiten $\Phi_i(t-1)$ aller Zustände i des Modells M zum vorangehenden Zeitpunkt $t-1$
- Die optimale Teilstruktur des vorangegangenen Schritts enthält auch die zur späteren Rekonstruktion der optimalen Zustandsfolge X erforderliche Information.

Initialisierung:
$$\Phi_1(0) = 1 \tag{38}$$
$$\Phi_j(1) = a_{1j} b_j(\mathbf{o}_1) \; \forall \; 1 < j < N \tag{39}$$
Induktion:
$$\Phi_j(t) = \max_i \{\Phi_i(t-1) a_{ij}\} b_j(\mathbf{o}_t) \tag{40}$$
$$\forall \; 1 < j < N, \quad 1 < t < T+1$$
Terminierung:
$$\Phi_N(T+1) = \max_i \{\Phi_i(T) a_{iN}\} \tag{41}$$

- Größte Wahrscheinlichkeit $\hat{P}(O|M)$:

$$\hat{P}(O|M) = \Phi_N(T+1) = \max_i \{\Phi_i(T) a_{iN}\} \tag{42}$$

- Logarithmierte Wahrscheinlichkeiten verwenden, um Multiplikationen mit sehr kleinen Zahlen zu verhindern:

$$\begin{aligned} \Psi_j(t) &= \log \Phi_j(t) \\ &= \max_i \{\Psi_i(t-1) + \log(a_{ij})\} + \log(b_j(\mathbf{o}_t)) \end{aligned} \tag{43}$$

- Veranschaulichung: Optimalen Pfad durch Digraphen finden
  - Vertikale Achse: Zustände j des Hidden Markov Modells
  - Horizontale Achse: Nummer t des Sprachvektors (also Zeitachse)

© R.-R. Grigat 2003-2019

- Knotenwichtungen entsprechen log-Wahrscheinlichkeiten $\log b_j(o_t)$, dass ein bestimmter Sprachvektor o_t zu der entsprechenden Zeit beobachtet wurde
- Kantenwichtungen entsprechen logarithmierten Übergangswahrscheinlichkeiten
- Die log-Wahrscheinlichkeit jedes Pfades ist die Summe der Übergangswahrscheinlichkeiten (Kanten) und der Ausgabewahrscheinlichkeiten (Knoten)
- Anmerkung: Für die Worterkennung werden nur die optimalen (größten) Wahrscheinlichkeiten benötigt, der optimale Pfad muss hier nicht konstruiert werden

0.16.3.10 Veranschaulichung des Viterbi-Algorithmus

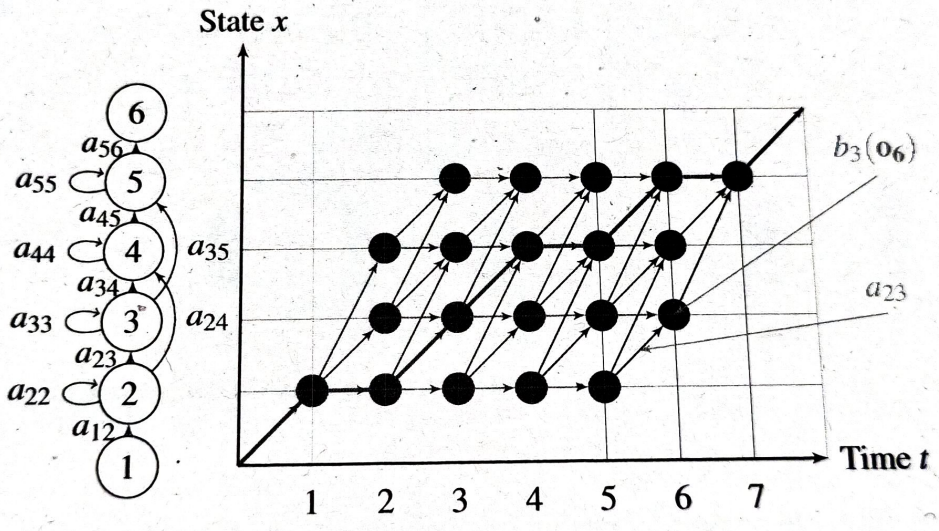


Abbildung 136: Viterbi-Algorithmus für Worterkennung

© R.-R. Grigat 2003-2019